The Session Manager Field Manual

A tab-by-tab operator's guide to Claude Code Session Manager — what each surface is for, the one thing people get wrong about it, and the workflow that makes it pay for itself.

Version 1.0.0 · released 2026-08-07 · documents Session Manager v0.64.0

Contents

Getting Started — the 10-minute setup

Session Manager is a local cockpit for the Claude Code CLI. It does not replace `claude`; it wraps it — giving you a terminal, a scheduler that runs work while you sleep, and roughly two dozen configuration and observability surfaces over the same `~/.claude/` directory the CLI already reads.

1. Install and launch

There is nothing to build. The app ships on npm and runs straight from `npx`:

```
npx claude-code-session-manager@latest
```

First launch recompiles the native terminal backend (`node-pty`) for your Electron runtime, so give it a minute. Linux and macOS only.

If you want the stripped-down version — one terminal, no tabs, no config surface — add `--simple`:

```
npx claude-code-session-manager@latest --simple
```

figure pending capture: app boot, Home tab, empty project list

1 The left nav — every surface lives here. 2 **Open / Start Project** — the only way a project enters the app. 3

The Almanac footer: connection dot, 5-hour usage, active branch, app version.

2. The one idea to internalise: TAB = project, EPIC = unit of work

Almost every mistake new users make comes from missing this. Session Manager has exactly two levels of scope, and they are not interchangeable:

Concept	What it actually is	How many
TAB	A working directory. The cwd *is* the project identity — everything the app stores for a project keys off that path.	One per project. Picking an already-open project re-activates its tab; it never opens a second.
EPIC	One small unit of work inside that project, backed by its own tagged Claude session.	Many per tab. Short-lived by design — one Epic per thing you're doing.

> **The rule that follows:** if you want a second conversation in the same project, you want a second *Epic*, not a second tab. Tabs are for switching projects.

3. Where your data lives

When a tab first opens on a directory, Session Manager creates one folder inside it:

```
<your-project>/session-manager-operations/
├── prompt-sessions/   # your Epics — active index + archived Epics
├── scheduler/         # PRDs, queue, and run state
├── project-brief/     # the Home tab's project brief
└── logs/
```

It is plain JSON and Markdown, it sits inside your repo, and it is meant to be read by humans and committed if you want it committed. Nothing about your project is stored in a cloud service by this app — the marketing page's "local-first" claim is literal.

4. Your first five minutes

1. **Open / Start Project** in the left nav, and pick a repo you actually work in. A native folder picker opens; "New Folder" starts a fresh one.
2. The Terminal tab boots a real `claude` session in that directory. This is the same CLI you already use — same settings, same skills, same MCP servers.
3. Press **New Epic**. Pick an *Agent* (who works) and a *Mission* (feature / bug / discussion), type what you want, and submit.
4. Watch it run. That's the loop. Everything else in this manual is about doing that loop faster, cheaper, and unattended.

5. What to read next

If you only read two more chapters, read Epics & Sessions (so you stop losing context) and Scheduler (so the app starts working while you don't).

Epics & Sessions — Terminal and Chat are one session

An Epic *is* a tagged Claude session. Not a folder of sessions, not a wrapper around one — a 1:1 relationship. Once that clicks, the two views over it stop being confusing.

One session, two views

Every Epic mints exactly one `claudeSessionId` when it is created. Both ways of talking to it attach to that same id:

View	What runs underneath	Best for
Terminal	An interactive PTY: `claude --resume <sessionId>` in the Epic's cwd.	Anything needing approvals, interruptions, or you watching closely.
Chat	Headless: `claude -p --resume <sessionId>`, streamed back as a feed.	Long unattended turns, and reading back what happened.

> **Switching views hands the session over — it never forks it.** Attachment is mutually exclusive: exactly one view holds the session at a time. Context continuity across the switch is the whole point, so if you ever see a fresh empty session after switching, that's a bug worth reporting, not expected behaviour.

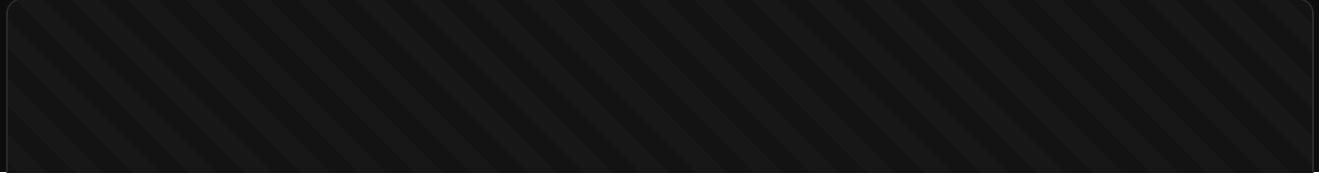

figure pending capture: Epic detail with the Terminal/Chat view toggle visible

1 The view toggle — one session, two renderings. 2

The verbosity dial (1 loudest → 5 quietest). 3

The Epic's event chain: prompt → PRD → response.

Creating an Epic: two independent choices

The New Epic card asks two separate questions, and it is worth being deliberate about both:

1. **Agent — who is working.** A persona from your Agent Library (`~/.claude/agents/*.md`). This also decides which model the session launches with: the persona's own `model:` field wins, falling back to your global default.
2. **Mission — how they work.** One of *feature*, *bug*, or *discussion*. This sets the grounding template the session opens with, and how eagerly the agent should decompose work into scheduled PRDs.

Those two, plus a one-line summary of what's actually grounding the session, compose the opening prompt in a fixed order — **Actor, Input, Mission**, then your own goal text. That's the AIM framework, and it's why an Epic tends to start productively instead of spending three turns establishing context.

The "advanced" flip: the grounding board

Flip the New Epic card and you get an inventory of what this session will really load — the CLAUDE.md files, skills, MCP servers, and hooks it will discover on disk, plus your git branch and the other Epics in the project. It is an **inspector, not a control panel**: there is no toggle to make the CLI skip a file it would otherwise find. Read it to understand why an agent behaved a certain way.

Lifecycle: three states, one transition you care about

```
proposed ──(Approve & start, or submitting the first prompt)──▶ active ──▶
```

- **proposed** — reserved but not started. *Nothing has been spent yet.* Every Epic is born here.
- **active** — the session is running, authoring PRDs, receiving scheduler updates.
- **completed** — archived to its own JSON file. The session id is dead; resuming mints a new one that traces back.

> **Don't repurpose an Epic.** Its title and objective are fixed for the life of its session — that objective *is* the first prompt. Iterate in follow-up messages; when the unit of work changes, start a new Epic. The rename action in the row menu is for fixing a typo, not for changing what an Epic is about.

Where per-task behaviour belongs (and where it doesn't)

This is the most common architectural mistake users make. Settings — including Permissions, Hooks, MCP Servers — edits *on-disk files that the CLI reads on every single invocation*. A Project-scope Settings change alters the substrate for every current and future Epic in that project. It is not per-conversation curation and never was.

You want…	Correct lever
This kind of task to behave differently	A custom **Mission tag** (its prompt template)
A different reviewer/implementer style or model	A custom **Agent persona**
Different permissions, hooks, env vars, MCP servers	**Settings** — genuinely substrate

Reading a session without drowning

The Chat feed has a five-level density dial, numbered loudest-first: **1 raw**, 2 detail, 3 standard, 4 brief, **5 summary**. Tool cards appear only at levels 1–2. Nothing is ever discarded from the store — raise the level and the byte-exact record comes back with no re-read.

Two things ignore the dial on purpose, and you should know why:

- A turn **asking you something** is never hidden or clamped at any level. Hiding a question strands a run waiting on an answer nobody can see.
- The **in-flight** bubble keeps its tool strip at every level — mid-run it's the only progress signal there is.

Scheduler — author once, run around your token window

The Scheduler is the feature that pays for the app. You write a PRD, it lands in a queue, and Session Manager runs it as a headless `claude -p` job timed around your plan's rolling 5-hour token window — auto-pausing when you're rate-limited and resuming the moment the window resets.

What the Scheduler is a browser over

It is a viewer and operator surface on one hierarchy, scoped to the active tab's project by default ("All projects" is the explicit escape hatch):

```
TAB (project)
└── EPIC (unit of work)
    └── PRD (one scheduled job)
```

On disk, PRDs live at `<project>/session-manager-operations/scheduler/epics/<epic-id>/prds/`, with the queue and history sharded

alongside in `scheduler/state/`. Every PRD belongs to an Epic — there is no such thing as a free-floating PRD.

figure pending capture: Scheduler tab, Queue view, at least one running job

1 Queue / PRDs / History — three views, one panel. 2 Mode selector: manual · on-reset · when-available. 3 A job's status badge — *needs_review* is a question, not a failure.

The three run modes

Mode	Behaviour	Use when
`manual`	Nothing runs until you press go.	You're debugging the queue itself.
`on-reset`	Fires at each 5-hour window reset.	You want a predictable burst, not a trickle.
`when-available`	Polls your billing usage every 60 s and runs whenever there's headroom. **The default.**	Almost always.

Concurrency: one pool, machine-wide

Scheduler jobs and Chat runs draw from the *same* pool of concurrent `claude -p` sessions (default 5, adjustable from the Home tab). On top of that sits a memory gate: a job won't start if the machine doesn't have headroom for it. That gate is machine-

aware rather than a hardcoded job count, which is what makes running five parallel agents on a laptop survivable.

Writing a PRD that actually finishes

Two real incidents shaped every rule here, and both are worth internalising:

The poll-hang

A PRD used `until $(curl … | jq .uptime)` to wait for a deploy. The condition was unsatisfiable — that deploy type never restarts the API — so the job spun for **2 h 47 m** until the watchdog killed it.

Rule: never write an unbounded wait. Every loop gets a maximum iteration count and an explicit give-up branch.

The post-AC overrun

A PRD declared success, then launched a fixture generator over 50,000 seeds that appeared nowhere in its acceptance criteria. It ran **2 h 44 m** before a human killed it.

Rule: the acceptance criteria are the *ceiling*, not the floor. When the AC are met, stop.

The checklist

1. One PRD, one outcome. If you can't state the outcome in a sentence, it's two PRDs.
2. Acceptance criteria must be checkable **headlessly**. "Launch the app and click through the tabs" cannot be verified by a `claude -p` job — it has no GUI and will be SIGTERM'd.

3. Every loop is bounded. Every wait has a timeout.
4. Ordering comes from `dependsOn` — nothing else gates a batch.
5. Never reuse a PRD number.

When a job parks in `needs_review`

`needs_review` is **a question, not a result**. When a job lands there, Session Manager writes a deterministic root-cause report next to that run's own log and appends it as a response on the Epic that authored the PRD — so the question comes back to the conversation that asked it, not to a global inbox you'll never check.

This is also the strongest argument for writing tight PRDs: the clearer the PRD, the less there is to route back to you.

Self-healing you don't have to think about

- **Boot reconciliation** — on startup, jobs whose processes died are reaped and classified (success / timeout / error). A still-alive orphan is terminated gracefully first so its log isn't misread mid-write.
- **Dead-process reaper** — periodic PID-liveness checks catch hung jobs.
- **Supervisor** — every 15 minutes, a probe per running job; it kills a stuck poll-loop's shell without killing the agent that spawned it.
- **Definition-of-done gate** — when the queue drains, completed PRDs are re-verified against their own acceptance criteria and a report is written. Idempotent, so a re-drain over the same set is a no-op.
- **External watchdog** — an out-of-process timer relaunches the app if its heartbeat goes stale while jobs are outstanding.

> All of it exists so that "queue it and go to bed" is a real workflow rather than an aspiration.
